

LearnedCache: An eBPF-Integrated Perceptron-Based Eviction Policy for the Linux Page Cache

Jayden Qi

Abstract—Linux is the foundation of the digital age, accounting for the majority of the cloud and mobile OS markets. Any device that runs Linux uses the Linux page cache, a central pillar in OS and application performance, serving to reduce extraneous disk access. Many page cache eviction policies have been developed but remain bound by the rigidity of heuristics. The rise of AI-driven tools in recent years, melded with the ever-increasing variety of workloads for Linux devices, sets the stage for machine-learning-driven cache eviction policies. Promising research has been done in this field, but only in the field of userspace applications such as CDNs. We develop LearnedCache, an eBPF-integrated single-layer perceptron-based cache eviction policy for the Linux page cache, trained on real kernel data from diverse workloads. We demonstrate median AUCs of near 80% over multiple linear models modeling page reuse time, then take a step further by embedding these models inside the Linux kernel for real-time performance evaluation. Through statistical testing over 50 paired trials against a baseline of FIFO for each workload, LearnedCache reveals that machine-learning derived cache eviction policies are practical in the Linux kernel under representative empirical workloads and are able to surpass conventional FIFO by statistically significant margins of up to 10% in insertion ratio—a frequency-adjusted derivation of cache hit rate—in specific workloads while incurring minimal overhead.

I. INTRODUCTION

Linux serves as the bedrock for modern digital infrastructure, making up an estimated 90% of the public cloud across AWS, Azure, and GCP—spanning from web servers to specialized supercomputers to compute for groundbreaking AI today. [1] Linux also had a firm hold in the mobile device market, serving as the kernel for an estimated 3.9 billion Android devices worldwide. [2] Yet, with this dominance, the adaptation of custom page cache algorithms beyond those shipped in the kernel itself has been slow to follow, creating a longstanding bottleneck that, if minimized, would significantly reduce cache misses, disk writes, and potentially offer large performance gains. Dating back to 1981, Michael Stonebraker corroborated the need for eviction policies beyond the one-size-fits-all all LRU/MRU-derived algorithms, proclaiming that these could not possibly accommodate the growing variety of database workloads back in the day. [3]

In the modern day, this fact still holds—the Linux kernel still uses a LRU derivation, namely Multi-Generation LRU, even across the even wider spread of workloads given to

it. [4] One can attribute this lack of adaptation to one of accessibility—up until recently, there existed no high-performance solution to insert a custom eviction policy into the page cache without requiring kernel patches, holding a high skill barrier to entry. Previous works, such as the recent P2Cache, have tried to create such a framework, but in doing so impose restrictions on implementation, such as the limitation to a single LRU/MRU queue. [5] `cache_ext`, created near the end of 2025, introduces a framework where an eBPF program (also a relatively new technology) can interface with their provided custom kernel to create “hot-swappable” custom cache eviction policies with near-kernel level performance. [6] With the use of eBPF, sandboxed user-written programs are algorithmically verified for unsafe behavior when run in kernel space. [7] However, the policies offered by this paper and those that remain the de facto standard remain limited to heuristics—still being variations of LRU, MRU, and FIFO.

A lightweight machine learning based eviction policy would theoretically be able to adapt to any workload and outperform traditional policies, being able to create a highly specific eviction policy for any workload given trace data to train on. An ML-based eviction policy would also save time and effort in creating tailored algorithms for the wide variety of workloads present in the cloud and other infrastructure. Like with the problem of developing a way to allow easier creation and swapping of policies, works have attempted to create machine learning solutions to cache eviction: ML-CLOCK outlines a possible perceptron-based eviction policy for page caches on a “trace-driven simulation”, but doesn’t publish the specific details, or the simulation used. [8] Yang et al. offers performant ML-based eviction policies by adding an ML model on the tail end of a heuristic algorithm, [9] and Song et al. in HALP [10] introduce a trained eviction policy for the YouTube CDN, achieving a 9.1% reduction in byte miss ratio.

Although ML-driven cache eviction policies show potential, no research thus far has trained one on then placed it into the Linux page cache, most likely due to the tooling constraints.

LearnedCache’s design is driven by these insights and the hurdles encountered along the path to kernel-level implementation. Because of the proof that ML-based application-informed policies are able to perform well in other contexts, we extend this to an extremely vital part of computing—the kernel page cache itself. We prove that the

training of high-performance models on eBPF trace data is possible, and also prove that the embedding of said models and the carry-on of their performance to the Linux page cache itself is practicable through eBPF [11], albeit with challenges. We show that it is possible for such a neural eviction policy to significantly outperform a baseline of FIFO in specific workloads.

The goal of our work, thus, is to demonstrate the following through LearnedCache:

- Accurate and performant neural models can be trained using kernel trace data of varying workloads.
- The deployment of an eBPF-driven implementation of said models is possible through `cache_ext`.
- These models can still show significant improvement over heuristic policies in a kernel context.

We provide the code and artifacts for these contributions:

- A `cache_ext` based eBPF tracer to log meaningful data for the design, training, and initial evaluation of a model.
- A Python module and notebooks processing the said datasets and implementing a discrete perceptron-based Bradley–Terry linear pairwise ranker over page reuse times.
- A `cache_ext` eBPF implementation of this model embedded into the kernel itself, and evaluation of its performance.

II. BACKGROUND AND LIMITATIONS

The following is partially adapted from the `cache_ext` background section.

A cache is a high-speed, temporary transport-layer storage that stores copies of frequently accessed data to improve access times. Operating systems use a page cache to store fixed-size chunks (usually of 4KiB or 4096 bytes), known as pages, of files on disk into much faster random access memory (RAM) for faster repeated reads and writes. [12] Because the amount of free RAM a computer has is (in the majority of instances) far less than the amount of disk space, reads of multiple files from disk will fill the page cache up quickly, requiring eviction, or removal, of pages from the cache. The algorithm that decides which pages to evict from the cache is called an eviction policy.

Linux ships with two main eviction policies inside the kernel: LRU (Least recently used) and the more modern MGLRU (Multi-generation least recently used) [4]. They are alike in that they both store lists of pages present in the page cache and always choose the least recently used (last used, longest time ago) pages to evict.

A. Folios

Linux has recently begun transitioning to folios in place of pages. [13] A folio is either a zero-order page, or a group of pages contiguous in memory that can be treated like a singular larger page. The terms folio and page will be used interchangeably, and all folios mentioned are equivalent to a single page.

Modern operating systems, such as Linux, are separated into two main levels of privilege—kernel space is where the kernel, responsible for interacting with the CPU, RAM, disk, and other devices, runs with high privilege. [12] The Linux page cache and its eviction policies reside in kernel space. In contrast, user space is a restricted environment where user programs run with access to less instructions. To interact with the devices that the kernel is responsible for, a userspace program must request it from the kernel itself.

There are more practical details between user-space and kernel-space programs. The first is development. The development of applications is mostly done in and for userspace. To modify kernel-resident behavior, a developer historically has needed to modify the kernel itself.

The development of eBPF and its integration into the Linux kernel has shifted this truth. eBPF allows small sandboxed programs compiled to BPF bytecode to run in kernel space at specific points. To ensure safety, eBPF programs must be formally verified by the eBPF verifier, which imposes harsh limits on the size, memory usage, and complexity of programs. [7] Programs written for eBPF must not exceed a stack size of 512 bytes, [14] not have a total execution tree of over a million instructions, [14] and must have no possibility of out-of-bounds array accesses, [15] making the implementation of non-trivial algorithms and models in it difficult. Furthermore, the Linux kernel forbids the use of floating-point numbers or operations in kernel space, [16] further increasing the difficulty of implementation for machine learning models.

1) `cache_ext`: `cache_ext` is a recent eBPF-based framework created to allow implementation of page cache eviction policies inside the kernel by developers with minimal overhead. [6] User-created eBPF programs mount onto a series of hookpoints provided by the `cache_ext` kernel and manage pages through its own algorithms and lists. `cache_ext` provides a built-in trusted data structure to store lists of pages for convenience, as well as helper functions to traverse and modify these lists. All `cache_ext` programs are able to be bounded to only listen to a single file directory and the programs within a cgroup. LearnedCache utilizes this to collect workload-specific training data and for evaluation.

III. MODEL DESIGN AND IMPLEMENTATION

In this section, we show the engineering and design process for the model used in LearnedCache. On a high level, this is split into two parts: the tracer that collects kernel data and traces pertaining to the feature set of the model, implemented in both Rust and C, and the model itself, written in Python with scikit-learn, Keras, and TensorFlow. The model, at its core, must be able to be used to produce a relative time where a page will be next accessed, given information about its previous accesses. The data collected from the tracers must always be uniform for both model training, evaluation, and deployment—we use

a set of features related to both the spatial and temporal dimensions of page access events. We now describe the design of the model in detail, beginning with the tracers.

A. Filebench and Control

Tracing data collected while running Filebench is employed; Filebench is a synthetic filesystem workload generation and evaluation framework designed to be scriptable and model varying workloads for all of our model training, evaluation, and deployment. [17] This is due to the limited access we had to the array of technology used to run the variety of workloads that exist in the world today. Filebench offers a wide variety of pre-made and customizable workloads that simulate the actions of real cloud machines, which, after slight modifications for reproducibility, act as “best-fit” approximations upon which LearnedCache is built.

`cache_ext` provides the option to run in a cgroup (a hierarchical group of processes bound by memory and CPU), and this is employed for control. [6] An option for a watch directory to which the policy solely listens is also provided and used; all filebench workloads and eviction policies run in cgroups, and filebench is set to utilize a disk-backed `/tmp` directory while the policy listens to it. Furthermore, because of the already present install scripts by `cache_ext` for Ubuntu, all policies are run on a virtual machine with KVM passthrough for isolation.

B. Tracer Design

`cache_ext` programs are exposed to a `struct_ops` interface, creating functions that run on cache eviction requests, eviction itself, insertion, and access. [6] Notably, access hooks onto the kernel function `folio_mark_accessed`, which is called by the kernel to mark a folio as recently accessed. [18] This is used by LearnedCache to get page access datapoints. This implies, however, that no absolute measure of access count can be used to collect training data.

Initial feature sets utilizing absolute access counts were discarded as they contained little meaningful information and were temporally dependent. Features such as the number of accesses to a page could not be used because they were dependent on when the tracer was started, and could therefore not be generalized to other runs at arbitrary times. Simply comparing the time until access to pages was not enough; all decisions must be framed during eviction time in order to meaningfully compare the reuse times of pages. After iteration and testing with a quickly developed neural network, the following final featureset was settled upon and implemented as a tracer on top of `cache_ext`’s FIFO policy.

- Delta time between last accesses of page.
- Delta time between last 2 accesses of page (ie, second and third to last).
- Delta time between last access of inode.

- Delta time between last 2 accesses of inode (ie, second and third to last).
- Distance from last access within file.
- File size in pages.
- Relative exponential moving average for page accesses.
- Relative exponential moving average for inode accesses.
- Time from last access to eviction.

Where the exponential moving averages are calculated as follows: a score is stored for each folio, which is incremented on each access. This score discretely decays with a half-life of 1 second.

C. Tracer Implementation

For both the preliminary and the final `cache_ext` tracer, the same approach is used to derive these logged metrics at each access event. An eBPF map with a high limit of entries containing all the necessary information is stored, and features are derived from it while updating it at each insertion and access event. A combination of device, inode, and offset (page index) was chosen as the key for these maps because a single folio pointer itself might not point to the same memory location nor be valid for its lifetime. Therefore, these three values, which can be easily obtained from a folio pointer, are unique and hold more meaning to a model.

Training/testing datasets for each workload were created from combined eviction timestamp and page access event traces. Each eviction event is matched to the closest access event that occurred before it for each page, to simulate the time from access to eviction.

An entry is initialized or reused upon insertion, and the required metrics are derived upon access. One caveat here is what to do when a page with fewer than 2 reaccesses must be logged, as then one or both of the time deltas will not be calculated. The solution is to force it to the largest value possible to be stored in a 64-bit integer, aligning with the data preprocessing pipeline used for feature discretization, shown later.

The tracers, as well as the machine learning policies shown later, are run using a modified version of the test harness that the original `cache_ext` authors used, written in Python. The resulting tracer output files are saved using a compact binary format to the disk in a directory not watched by the tracer nor touched by filebench. The data is then transferred to the host machine and parsed into a plaintext comma-separated values (CSV) file. Figure 1 summarizes this end-to-end tracing and evaluation workflow, from the host machine to the virtualized kernel environment.

D. Model Design

The machine learning model comprises two main components: the preprocessor and the ranker. The model is a pairwise linear ranker implemented as a perceptron. This model architecture was chosen to maximize performance as it boils down to a vector dot product between the feature

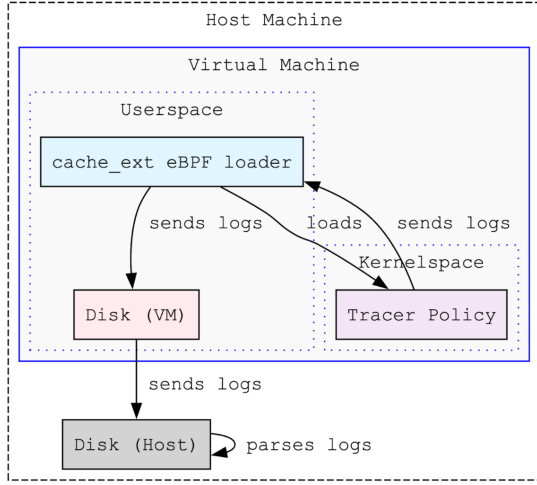


Fig. 1. Tracer and evaluation harness workflow.

vector and the weight vector, minimizing the number of operations when being run multiple times in the kernel. The entire model was built using scikit-learn, Keras, and TensorFlow.

1) *The Discretizer*: Early on, the decision was made to discretize the logs into “bins” based on quantile, capped at a set number of 10 before feeding it into the model. Two main benefits stem from this: the first is data uniformity. As shown in Fig. 2 from the initial evaluation of a tracer run on the **fileserver** filebench workload, both time delta and EMA statistics are heavily left-skewed, even when viewed on a logarithmic scale.

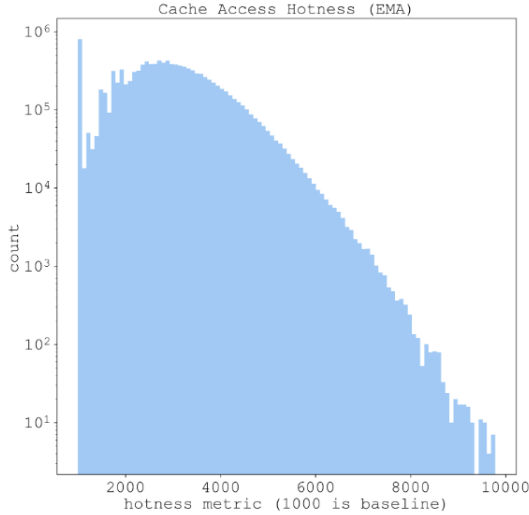


Fig. 2. Skewed feature distributions motivating discretization.

By binning, near-uniform distributions of training data were achieved. It is to be noted that for data without much variation, the discretizer could not create the maximum number of distinct bins. Consideration was taken on the amount of information loss that discretization could bring,

but the gain was perceived to be more valuable than the possible loss. The second important advantage gained by binning of input data is that it allows for one-hot encoding in the model, giving the model power to generalize to non-linear relationships between features and reuse time.

2) *The Model*: The model itself is created as a Bradley–Terry [19] pairwise ranker over one-hot encoded cache reuse times. The following descriptions of the Bradley–Terry model are largely adapted from Fan’s Stanford lecture notes on the topic. [20] A Bradley–Terry model is a statistical probability model used to discern the probability of candidate A winning over candidate B given their respective strength scores. It can be defined as the sigmoid function of score differences. Its conversion to a perceptron over feature differences is shown below.

$$w \in \mathbb{R}^d, \quad x_A, x_B \in \mathbb{R}^d, \quad \beta(\cdot) \in \mathbb{R}, \\ \beta_A = \beta(x_A), \quad \beta_B = \beta(x_B), \quad p_{A \succ B} \in (0, 1),$$

A sigmoid is defined as such:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (1)$$

The Bradley–Terry model is defined as such, where β_A and β_B are each candidate’s scores and $p_{A \succ B}$ is the probability that A is ranked above B :

$$p_{A \succ B} = \frac{e^{\beta_A - \beta_B}}{1 + e^{\beta_A - \beta_B}} \quad (2)$$

A page’s score is the dot product of transposed perceptron weights and input features:

$$\beta(x) = w^\top x \quad (3)$$

Thus, the Bradley–Terry model can be used to compare two pages:

$$p_{A \succ B} = \sigma(\beta(x_A) - \beta(x_B)) = \sigma(w^\top x_A - w^\top x_B) \quad (4)$$

By the distributive property, the perceptron can operate over feature differences:

$$p_{A \succ B} = \sigma(w^\top (x_A - x_B)) \quad (5)$$

Therefore, a large number of pairs is sampled from eviction events reconstructed from access and eviction logs, and the model is trained on the differences between the feature vectors. In practice, the sigmoid function serves only as a gradient for backpropagation during training. Because of the linearity of the model, using it to rank N items is minimum $O(N \log N)$ due to the $O(N)$ algorithm finding individual scores for each item and running a $O(N \log N)$ sort over the scores. Although this is the case in theory, N is bounded by the number of features and the maximum number of candidates we try to sort for eviction, making it able to be deemed constant time.

IV. MODEL TRAINING AND EVALUATION

In this section, we show the training hyperparameters used to train the created model across workloads and their evaluation metrics. We answer the question of whether it is possible to train high-performing, efficient models based on kernel trace data collected during real workloads.

During training, a train-test split of four workload runs to one workload run was established for the trace data of each workload. The maximum number of epochs was set to 50 during training, with a batch size of 256. Early stopping was set with a patience of 5 epochs, restoring the weights to the most performant epoch after patience runs out. The optimizer used for models was Adam. [21]

Separate models were trained on each member of the following sample of workloads from Filebench, roughly stratified based on type of workload: **openfiles** was chosen as a metadata-heavy workload; **webserver**, **webproxy**, and **varmail** as cloud workloads; and **mongo** as a database workload. Each workload was edited to be bound to a set number of file operations (around 50,000). These were chosen based on resource feasibility (videosever could not be run, requiring hundreds of gigabytes of storage) and operational scope (the filemicro suite was not used due to each workload only doing a few file operations).

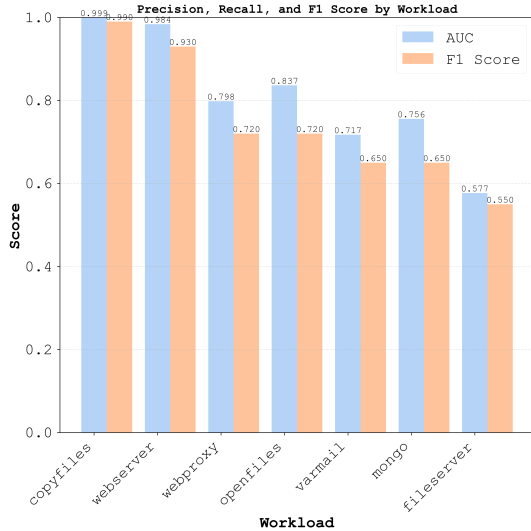


Fig. 3. Precision, recall, and F1 score by workload.

Figure 3 summarizes both AUC and F1 scores for each workload. From these metrics, a few observations can be made. The highest-performing models were **copyfiles** and **webserver**, achieving near-perfect AUC scores (0.999 and 0.984) and high F1 scores (0.990 and 0.930). A decline is then observed across **webproxy** and **openfiles**, both of which result in an F1 score of 0.720. The scores further dip for **varmail** (mail server) and **mongo**, both of which record F1 scores of 0.650. The lowest performance is seen in the **fileservers** workload with an F1 score of 0.550. It is also notable that across all tested workloads, the

AUC consistently exceeds the F1 score. Ultimately, it is concluded that accurate linear neural models can be trained using real trace data directly from the kernel. We will now interpret key observations in the data collected during training and evaluation.

A. Weight Balance

A benefit of a linear and one-hot encoded model is the transparency of the weights and their implications. Figures 4–6 show the learned weights of three representative models, each with different inferences to be drawn.

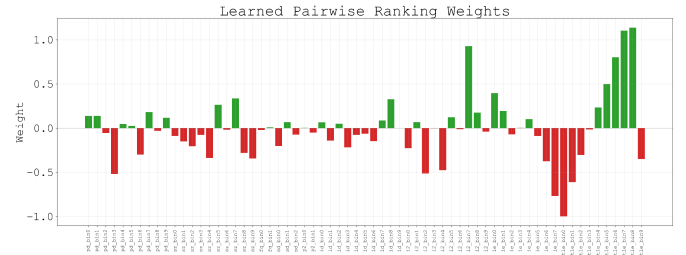


Fig. 4. Learned weights for varmail (green=keep, red=evict).

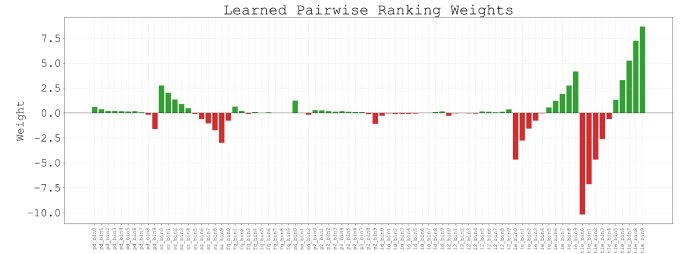


Fig. 5. Learned weights for **webserver** (green=keep, red=evict).

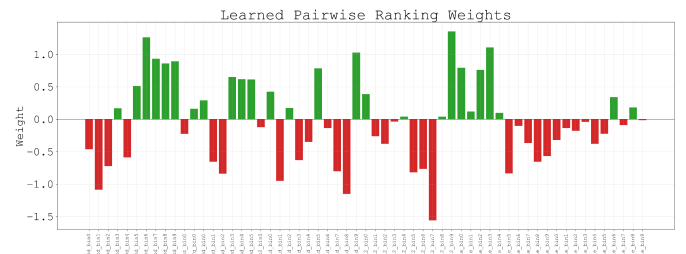


Fig. 6. Learned weights for `openfiles` (green=keep, red=evict).

These models show that page-reuse time varies across workloads. The observation can be made that for **webserver**, a policy based on the combination of file size (*sz*) and inode hotness would have practically the same effect as the **LearneCache** model for it. For **openfiles** and **varmail**, however, the weight distribution seems much more stochastic. This visualization of weights is useful practically as a model that can be generalized solely down to a few features is effectively the same as a heuristic policy

operating on those features. Most importantly, however, is that this serves as proof that models can generalize very well to workload-specific patterns in kernel trace data.

V. POLICY DESIGN AND IMPLEMENTATION

In this section, we show the design and implementation process behind the actual ML-based policy used to further evaluate the models in real-time workloads. We aim to answer the following questions:

- Can a model used to predict page reuse time be practically placed into the kernel with eBPF and `cache_ext`?
- What are the limitations imposed upon such an implementation?

At a high-level overview, the policy is akin to ML-at-the-tail in that it reranks the tail end of a FIFO-based policy. This architecture was chosen for performance reasons: it is simply not feasible to rank all available pages with a model, as LRU runs in near-constant time while the ranker runs at $O(N \log N)$. Weights and discretizer edges are exported and loaded into the eBPF program, which has its own eBPF-friendly implementation of the ranker (perceptron) and the discretizer, which it runs on eviction candidates upon eviction request.

A. Model Export and Loading

Figure 7 displays the model loading flow. Each component will be described.

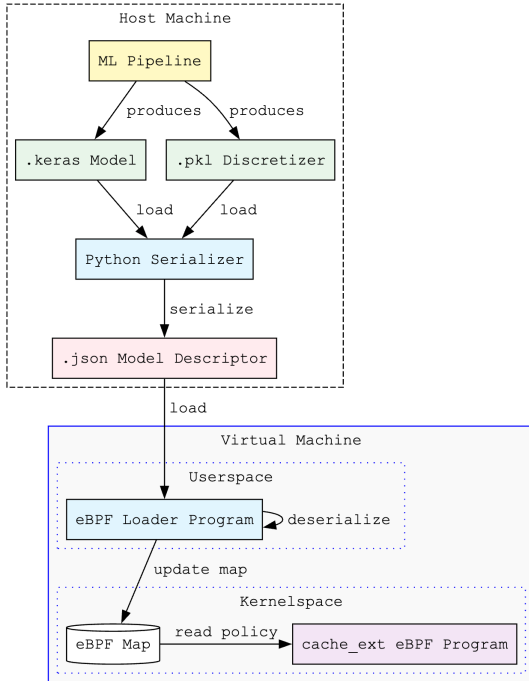


Fig. 7. Model export and loading flow.

A model for a specific workload can be defined by two things: the vector of weights for each one-hot encoded feature and the edges of the discretizer used to bin raw values. Therefore, the model training script exports both

the perceptron as a JSON object. The JSON object format is shown below, along with important caveats.

```

{
  "feature_names": [array of feature name strings],
  "n_features": number,
  "weight_scale": number (10000),
  "features": [
    {
      "index": number,
      "name": string,
      "n_bins": number,
      "bin_edges": [array of bin edge values, size = n_bins - 1],
      "weights_float": [array of float weights, size = n_bins],
      "weights_int": [array of integer weights (scaled by weight_scale), size = n_bins]
    },
    ...
  ]
}

```

For each feature, the file describes the number of bins, an array of edges, the weights for each input value corresponding to a single value of the enumeration created by one-hot encoding, and the quantized weights. There are three main considerations here.

Number of bins. While the number of bins is bounded by a maximum of 10, if a feature for a specific workload only takes on a few distinct values, this cap is not reached. Thus, the model can take in anywhere from 1–10 encodings per feature.

Quantization. eBPF programs are unable to do floating-point operations. To mitigate this, the model is symmetrically quantized without the use of a zero point to integer weights by multiplying the weight float values by the scale described in `weight_scale`, and then truncating to the nearest integer.

The eBPF userspace loader loads these weights and stores them in eBPF maps in order to be passed to the `cache_ext` eBPF policy in kernel space. The following are the source definitions for the maps, written in C.

```

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, NUM_MODEL_FEATURES);
    __type(key, __u32);
    __type(value, __u8); // n bins in each. cannot exceed MAX_BINS
} n_bins_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, NUM_MODEL_FEATURES);
    __type(key, __u32);
    __type(value, __u64[MAX_BINS]); // bin edges, [start, end)
} bin_edges_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, NUM_MODEL_FEATURES);
    __type(key, __u32);
    __type(value, s64[MAX_BINS]); // bin weights (integer, can be negative)
} nn_weights_map SEC(".maps");

```

Each eBPF map is of type array, meaning indices are not hashed. Maps must be used to store data because of the eBPF maximum stack size of 512 bytes, or 64 64-bit integers.

- **n_bins_map** stores the number of bins, or encodings, in each feature.
- **bin_edges_map** stores the bin edges for each feature. Note that the array is of a fixed size set to the maximum number of bins.
- **nn_weights_map** stores the weights for each feature, also with fixed-size arrays.

The maximum number of entries for all maps is set to the number of features present in the model.

B. FIFO

FIFO (first-in, first-out) is an eviction algorithm categorized by the use of a queue, usually implemented as a form of linked list. When a folio is inserted, it is pushed to the head of the list. At an eviction request by the kernel (with a max number of folios requested to be evicted set as 32 by **cache_ext**), the required number of folios is popped from the tail for eviction and returned to the kernel for eviction.

1) *Adaptation and Policy Design*: The core algorithm is a reranker over the tail of the **cache_ext**-provided sampling FIFO policy. [6] An ML-at-the-tail approach is adopted, inspired by the strategy of Yang et al. for userspace caches. [9] In the LearnedCache machine-learning-based eviction policy, oversampling by a factor of 5 folios is performed on the bottommost list, and the PID controller is not modified. The lowest-scoring folios (predicted to be reused last by the model) are then evicted.

Note: Although the model is trained as a pairwise ranker, its linearity means we can calculate an individual score for each folio. The sigmoid function is not implemented in practice—rather, it only exists to give the model a gradient during training.

Below is the code for model evaluation of a single feature, assuming the array **raw_features** contains the calculated non-discretized feature values.

```
#define PROCESS_FEATURE(feat_idx) \
do { \
    u32 idx = (feat_idx); \
    __u8 *n_bins_ptr = bpf_map_lookup_elem(&n_bins_map, &idx) \
    ; \
    if (n_bins_ptr) { \
        __u64 (*bin_edges)[MAX_BINS] = bpf_map_lookup_elem(& \
        bin_edges_map, &idx); \
        if (bin_edges) { \
            s64 (*weights)[MAX_BINS] = bpf_map_lookup_elem(& \
            nn_weights_map, &idx); \
            if (weights) { \
                __u8 n_bins = *n_bins_ptr; \
                if (n_bins > 0 && n_bins <= MAX_BINS) { \
                    __u8 bin = discretize_feature(raw_features[ \
                    feat_idx], *bin_edges, n_bins); \
                    if (bin >= MAX_BINS) bin = MAX_BINS - 1; \
                    score += (*weights)[bin]; \
                } \
            } \
        } \
    } \
}
```

```
} \
} \
} while (0)
#undef PROCESS_FEATURE
```

The definition for **discretize_feature** is as follows:

```
static inline __u8 discretize_feature(__u64 value, __u64 * \
    bin_edges, __u8 n_bins) { \
    __u8 n_interior_edges = n_bins - 1; \
    if (n_interior_edges > 0 && value < bin_edges[0]) return \
    0; \
    if (n_interior_edges > 1 && value < bin_edges[1]) return \
    1; \
    if (n_interior_edges > 2 && value < bin_edges[2]) return \
    2; \
    if (n_interior_edges > 3 && value < bin_edges[3]) return \
    3; \
    if (n_interior_edges > 4 && value < bin_edges[4]) return \
    4; \
    if (n_interior_edges > 5 && value < bin_edges[5]) return \
    5; \
    if (n_interior_edges > 6 && value < bin_edges[6]) return \
    6; \
    if (n_interior_edges > 7 && value < bin_edges[7]) return \
    7; \
    if (n_interior_edges > 8 && value < bin_edges[8]) return \
    8; \
    return n_bins - 1; \
}
```

Manual loop unrollings are performed to please the eBPF verifier. This, combined with using manual assembly guards, was found through iterations to be the simplest way to pass program verification without gambling on whether Clang decides to optimize away bounds that the verifier relies on or not. The core logic of the model boils down to two steps:

- Finding which encoding a given raw feature value falls under, then finding the weight of that specific bin.
- Summation of each weight found through this process.

Because one-hot encoding either produces inputs of 0 or 1, we are able to skip explicitly calculating the score for the Inactive bins for a feature, and simply add the weight of the active bin to the final score.

VI. POLICY EVALUATION AND ANALYSIS

In this section, we show the evaluation methodology and metrics for the LearnedCache machine learning cache eviction policy. We then visualize and interpret these results, offering significant evidence that high-performance models trained on kernel page cache trace data of specific workloads carry practical performance benefits when run in a kernel context.

A. Workload Evaluation Methodology

Each workload was evaluated except filesystem due to its low model performance. For each workload, 50 paired runs comparing naive **cache_ext** sampling FIFO and LearnedCache’s ML policy loaded with a model trained on the workload were run. Order randomization was utilized for control, meaning whichever workload ran first was decided by the odds of a fair coin flip. After trial completion,

TABLE I
WORKLOAD PERFORMANCE SUMMARY

Workload	% vs Baseline	Raw Change (Insertion Ratio)
webproxy	-9.69	-0.075
copyfiles	-8.78	-0.223
webserver	-3.76	-0.007
varmail	-0.08	-0.001
openfiles	+1.02	+0.049
mongo	+7.28	+0.225

the number of relative cache accesses and insertions was computed for each workload run and condensed into a single metric, which will be referred to as insertion rate, defined as

$$\text{insertion rate} = \frac{\text{insertions}}{\text{accesses}}.$$

Note that it is impossible to calculate the true page cache “hit rate”, as the metric used for cache accesses is a hotness metric, equal to the true number of accesses scaled by an unknown constant. This is an inherent limitation of the Linux kernel that all policies are subject to.

For each workload, a two-tailed paired t -test with a significance level of 0.05 is run over the differences between the model mean insertion rate and the normal mean insertion rate with `scipy.stats.ttest_rel` in order to determine statistical significance. [22] Defining μ_{policy} to be the mean insertion ratio of a workload under the policy, we define our hypothesis as:

$$H_0 : \mu_{\text{model}} - \mu_{\text{normal}} = 0$$

$$H_1 : \mu_{\text{model}} - \mu_{\text{normal}} \neq 0$$

95% confidence intervals are also created and visualized for each trial’s mean difference.

B. Workload Evaluation

Over the 5 workloads over which trials were run, **varmail** and **openfiles** showed statistically significant improvement, **randomread** and **copyfiles** showed statistically insignificant improvement, and **mongo** showed statistically significant regression. The key metrics are shown in Table I (bold indicates statistical significance).

The performance for each workload is visualized in Figs. 8–13. Note that checks for the 10% rule and random sampling are omitted as they are assumed to be true across each run.

1) *Webproxy*: Webproxy is a workload designed to simulate a proxy cache serving web content. A statistically significant improvement is observed. The p -value of 6.30×10^{-21} is far less than $\alpha = 0.05$, so we can reject the null hypothesis. A raw insertion rate decrease of 0.075 is found, with an effect size of -2.242 .

As shown in Fig. 8, there is a decrease in the mean difference of insertion rates. The distribution of model runs has noticeably less variation than that of normal FIFO runs. The histogram of paired differences is centered well below zero, indicating a consistent advantage for the

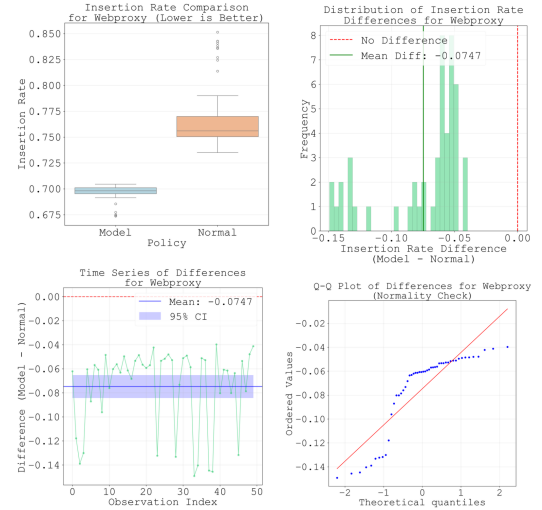


Fig. 8. Webproxy evaluation plots.

LearnedCache policy. The quantile-quantile plot shows some deviation in the tails, though the overall pattern remains reasonably close to linear. There are several worse-performing outliers for the model, but they are not enough to offset the overall improvement.

Over 50 trials, the model consistently outperforms the standard FIFO policy. The true mean difference appears to lie well below zero, and parity falls far outside of the constructed 95% confidence interval.

2) *Webserver*: Webserver is a workload intended to simulate a web server repeatedly serving webpages. A statistically significant improvement is observed. The p -value of 5.55×10^{-30} is far less than $\alpha = 0.05$, so we can reject the null hypothesis. A decrease in raw insertion rate of 0.007 is observed, with an effect size of -3.614 .

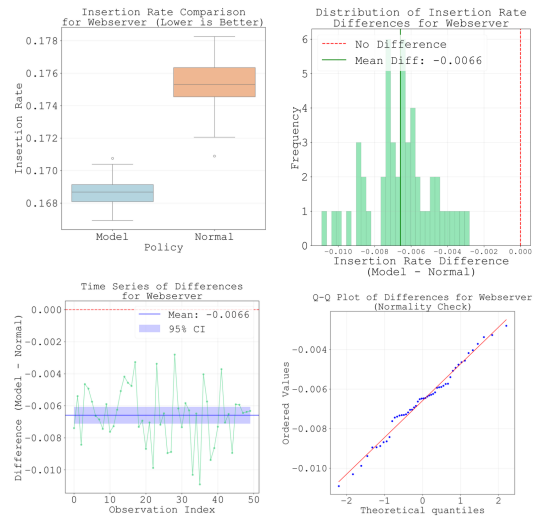


Fig. 9. Webserver evaluation plots.

As shown in Fig. 9, there is a clear mean decrease in

insertion rate from normal FIFO to the LearnedCache policy. Although the raw change is small in absolute magnitude, the spread of both policies is also very small, which leads to a very large effect size. The paired-difference histogram is entirely negative, and the time-series plot shows that all trials favor the model. The quantile-quantile plot is nearly linear, suggesting the normality assumption is reasonable.

Over 50 trials, the model consistently outperforms the standard FIFO policy. The true mean difference appears to lie below zero, and parity falls outside of the constructed 95% confidence interval.

3) *Copyfiles*: Copyfiles is a workload that repeatedly copies files from one directory to another. A statistically significant improvement is observed. The p -value of 1.68×10^{-2} is less than $\alpha = 0.05$, so we can reject the null hypothesis. A raw insertion rate decrease of 0.223 is found, with an effect size of -0.350 .

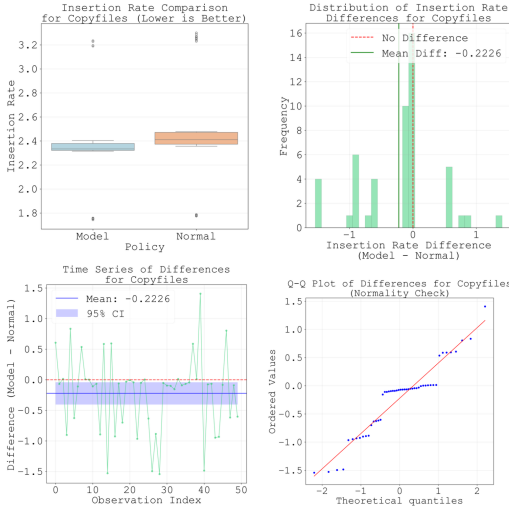


Fig. 10. Copyfiles evaluation plots.

As shown in Fig. 10, there is a moderate decrease in the mean difference of insertion rates of 0.223. However, the distribution of paired differences has high variance, which explains the relatively small effect size despite the larger raw change. The boxplots for the two policies are fairly close, and both show significant outliers at both ends. The quantile-quantile plot suggests noticeable departures from normality, especially in the tails, so the central limit theorem must be relied on to justify the test.

In over 50 trials, the model appears to outperform the standard FIFO policy more often than not, though with substantial variability across runs. Even so, the estimated true mean remains below zero, and parity falls outside of the constructed 95% confidence interval.

4) *Varmail*: Varmail is a workload that simulates a variable-file-size mail server. A statistically significant improvement is observed. The p -value of 8.95×10^{-3} is less than $\alpha = 0.05$, so we can reject the null hypothesis. A raw

insertion rate decrease of 0.001 is found, with an effect size of -0.385 .

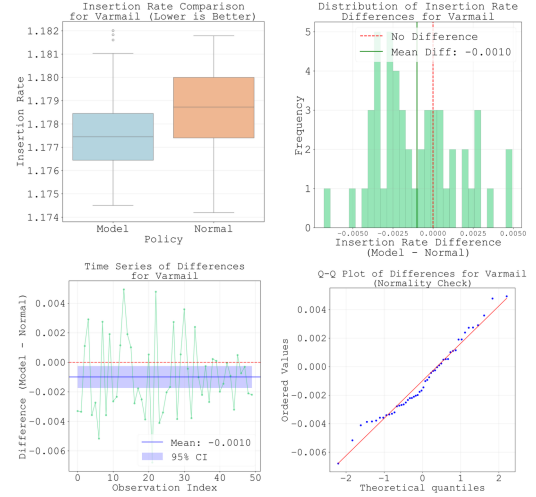


Fig. 11. Varmail evaluation plots.

As shown in Fig. 11, there is a slight decrease in the mean difference of insertion rates of 0.001. The distributions for normal and model runs overlap heavily, which is expected given the very small raw change. Still, the paired differences are centered slightly below zero. The quantile-quantile plot is fairly linear, so the normality assumption appears reasonable. There are a handful of mild outliers in both directions, but no severe departures are visible.

Over 50 trials, the model slightly outperforms the standard FIFO policy on average. The improvement is small, but the true mean difference still appears to lie below zero, and parity falls just outside of the constructed 95% confidence interval.

5) *Openfiles*: Openfiles is a small workload that repeatedly opens and closes files. No statistically significant difference is observed. The p -value of 8.18×10^{-1} is greater than $\alpha = 0.05$, so we fail to reject the null hypothesis. A raw insertion rate increase of 0.049 is found, with an effect size of 0.033.

As shown in Fig. 12, there is almost no mean difference in insertion rates between normal FIFO and the LearnedCache policy. The two boxplots have very similar centers and spreads, and the histogram of paired differences is broadly distributed around zero. The quantile-quantile plot is reasonably linear, suggesting that the differences are not strongly non-normal. There is no clear evidence of a systematic advantage for either policy.

Over 50 trials, the model and standard FIFO perform very similarly. The estimated true mean difference is close to zero, and parity lies well within the constructed 95% confidence interval.

6) *Mongo*: Mongo is a small workload designed to simulate small chunked accesses of a MongoDB-like database. No statistically significant regression is observed. The p -

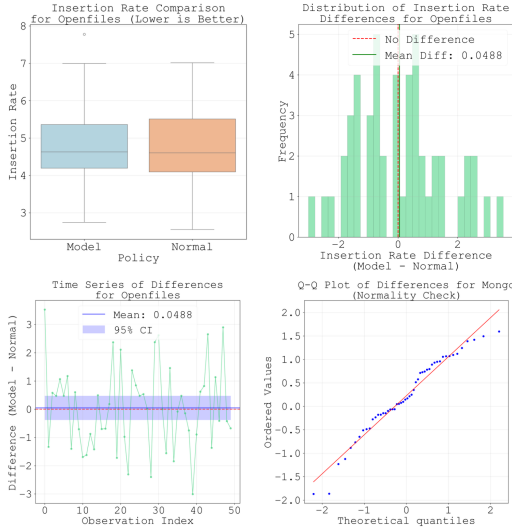


Fig. 12. Openfiles evaluation plots.

value of 5.99×10^{-2} is greater than $\alpha = 0.05$, so we fail to reject the null hypothesis. A raw insertion rate increase of 0.225 is found, with an effect size of 0.272.

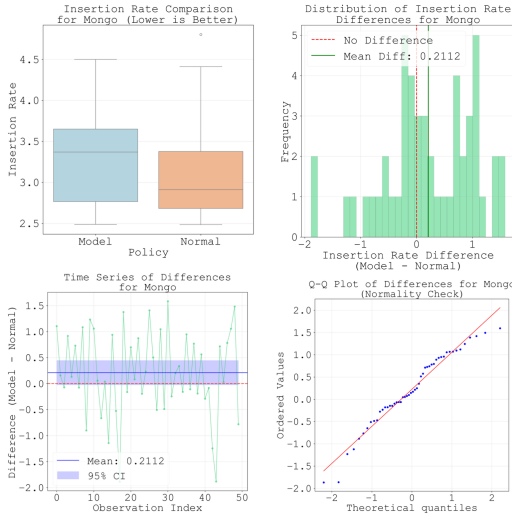


Fig. 13. Mongo evaluation plots.

As shown in Fig. 13, there is a noticeable increase in the mean difference of insertion rates of 0.225. The model runs appear to have somewhat greater variability, and the distribution of paired differences is centered above zero, suggesting worse average performance for the LearnedCache policy. The quantile-quantile plot is fairly close to linear overall, although there is some deviation in the tails. Because of the variance in the paired differences, the observed regression is not statistically significant at the $\alpha = 0.05$ level.

Over 50 trials, the model tends to underperform classic FIFO more often than it outperforms it, but not by a

statistically significant margin. The estimated true mean difference is slightly above zero, and parity remains within the constructed 95% confidence interval.

C. Latency Analysis

The use of the perceptron incurs overhead when making eviction decisions. Over 5,000 instances of max-capacity eviction requests, the median latency for model-assisted decisions was 25.34 microseconds, while the median for normal FIFO decisions were 22.17. The distributions for both, including means, are shown in Fig. 14:

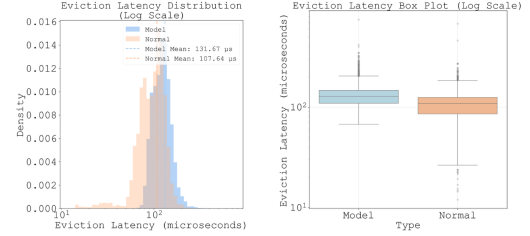


Fig. 14. Eviction decision latency comparison.

This difference is statistically significant.

RAM access is typically hundreds of thousands of times faster than reading from disk. Assuming the underlying machine matches these estimates and does not produce massive amounts of page cache churn, the time saved on the magnitude of milliseconds would far outweigh the model overhead.

VII. CONCLUSIONS

LearnedCache demonstrates that performant ML-based policies can be applied to environments as constrained as the Linux page cache, and see significant improvements in hits and overall file IO throughput. Offline models reach a median of 80% in AUC for pairwise ranking of page reuse time, and statistically significant improvements in insertion ratio are observed through the deployment of a novel eBPF and `cache_ext`-based eviction policy.

LearnedCache serves as a functional example for overcoming the hurdle of implementing perceptron-based models with eBPF in resource-constrained environments, yet also casts light on the fact that these models, while performing well in offline evaluation, might not behave as well when deployed in live kernel environments due to the inherent randomness of workloads. However, LearnedCache is not free from potential limitations. In order to effectively compare policies, static workloads ran with Filebench were bound to a set number of operations, thus creating limited variation that may not be completely generalizable to workloads truly running on production machines. The goal of LearnedCache is to provide a novel, executable solution for addressing the bottleneck caused by heuristic policies, and to take a step toward making ML-based page cache eviction applicable to billions of servers, mobile devices, and personal computers running Linux worldwide.

REFERENCES

- [1] UC San Diego Division of Extended Studies, “Linux cloud infrastructure (cse-41369),” UCSD Extended Studies, accessed: 2026-02-20. [Online]. Available: <https://extendedstudies.ucsd.edu/courses/linux-cloud-infrastructure-cse-41369>
- [2] CommandLinux, “Android global market share statistics,” accessed: 2026-02-20. [Online]. Available: <https://commandlinux.com/android/global-market-share-statistics/>
- [3] M. Stonebraker, “Operating system support for database management,” *Communications of the ACM*, vol. 24, no. 7, pp. 412–418, Jul. 1981.
- [4] J. Corbet, “The multi-generational LRU,” LWN.net, 2021. [Online]. Available: <https://lwn.net/Articles/851184/>
- [5] D. Lee, I. Choi, C. Lee, S. Lee, and J. Kim, “P2cache: An application-directed page cache for improving performance of data-intensive applications,” in *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage '23)*, Boston, MA, USA, Jul. 2023, pp. 31–36. [Online]. Available: <https://doi.org/10.1145/3599691.3603408>
- [6] T. Zussman, I. Zarkadas, J. Carin, A. Cheng, H. Franke, J. Pfefferle, and A. Cidon, “cache_ext: Customizing the page cache with ebpf,” in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*, Oct. 2025. [Online]. Available: <https://doi.org/10.1145/3731569.3764820>
- [7] B. Gbadamosi, L. Leonardi, T. Pulls, T. Høiland-Jørgensen, S. Ferlin-Reiter, S. Sorce, and A. Brunström, “The ebpf runtime in the linux kernel,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.00026>
- [8] MDPI Electronics, “Ml-clock (article page),” accessed: 2026-02-20. [Online]. Available: <https://www.mdpi.com/2079-9292/10/20/2503>
- [9] D. Yang, D. S. Berger, K. Li, and W. Lloyd, “A learned cache eviction framework with minimal overhead,” *arXiv preprint arXiv:2301.11886*, 2023. [Online]. Available: <https://doi.org/10.48550/ARXIV.2301.11886>
- [10] Z. Song *et al.*, “Halp: Heuristic aided learned preference eviction policy for youtube content delivery network,” in *Proc. USENIX NSDI*, 2023.
- [11] eBPF Community, “eBPF - Introduction, Tutorials & Community Resources,” <https://ebpf.io/>, 2024.
- [12] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.
- [13] J. Corbet, “The state of the page in 2025,” *LWN.net*, 2025. [Online]. Available: <https://lwn.net/Articles/1015320/>
- [14] The Linux Kernel Organization, “BPF design Q&A,” https://docs.kernel.org/bpf/bpf_design_QA.html, n.d., accessed: 2026-03-25.
- [15] —, “BPF verifier,” <https://docs.kernel.org/bpf/verifier.html>, n.d., accessed: 2026-03-25.
- [16] —, “Floating-point API,” <https://docs.kernel.org/core-api/floating-point.html>, n.d., accessed: 2026-03-25.
- [17] V. Tarasov, E. Zadok, and S. Shepler, “Filebench: A flexible framework for file system benchmarking,” *login Usenix Mag.*, vol. 41, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:56553130>
- [18] The Linux Kernel Organization, “folio_mark_accessed() — linux kernel source,” https://elixir.bootlin.com/linux/v6.6.8/A/ident/folio_mark_accessed, linux kernel v6.6.8; Accessed: 2026-03-25.
- [19] R. A. Bradley and M. E. Terry, “Rank analysis of incomplete block designs: I. the method of paired comparisons,” *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.
- [20] Z. Fan, “Lecture 24: The Bradley-Terry model,” STATS 200: Introduction to Statistical Inference, Stanford University, Autumn 2016. [Online]. Available: <https://web.stanford.edu/class/archive/stats/stats200/stats200.1172/Lecture24.pdf>
- [21] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [22] P. Virtanen *et al.*, “Scipy 1.0: fundamental algorithms for scientific computing in python,” *Nature Methods*, vol. 17,

no. 3, pp. 261–272, Feb. 2020. [Online]. Available: <http://dx.doi.org/10.1038/s41592-019-0686-2>